

COMS W4156 Advanced Software Engineering (ASE)

October 8, 2024

On this day ...



Nobel Peace Prize

2004 Kenyan [Wangari Maathai](#) is the first African woman to receive the Nobel Peace Prize for "her contribution to sustainable development, democracy and peace"



Environmentalist
[Wangari Maathai](#)

Agenda

1. Choosing Test Inputs
2. Equivalence Partitions and Boundary Analysis
3. Reminders



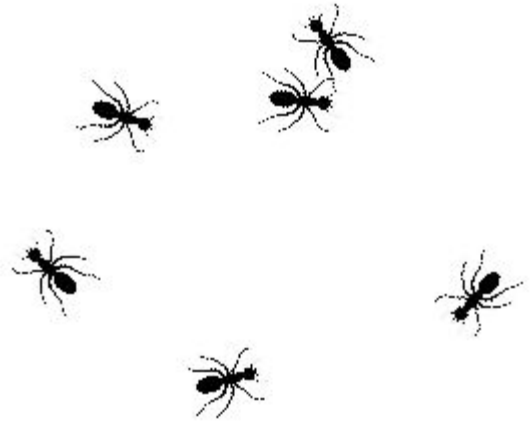
Baseline Testing Strategy

- *Test with one typical valid input, one atypical valid input, and one invalid input [preferably one invalid input value and one invalid input format]*

It's better than no strategy

It's better than the common CS1 strategy -
test program with two or three typical valid inputs
(happy path), if it didn't crash or hang it works!

Baseline applies to unit testing, integration testing,
system testing, API testing, ...



Where do Invalid Inputs come from?

Users

Network

Devices

Databases

Files

...

Other parts of
your codebase

Third-party
libraries

External services

Your test cases

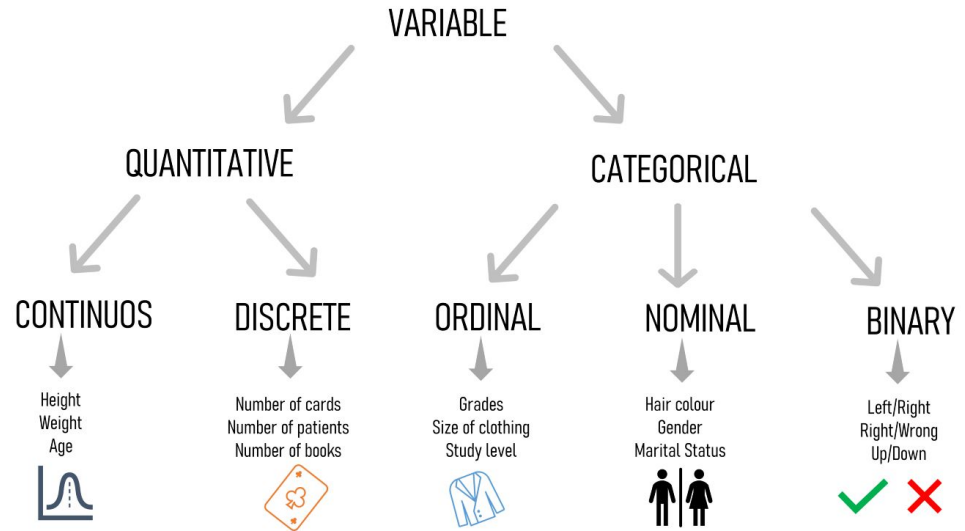
...



Better Approaches to Choosing Test Inputs

Test with a set of inputs that you think would make a nice demo

But keep in mind what might happen when the demo audience asks to “drive” - and intentionally tries bad inputs. If this is higher level management in your company or your venture capital investor, you cannot say No



How About Testing ALL Possible Inputs?

“Exhaustive Testing”

Usually “infinite” number of valid and invalid inputs

Even when number of valid and invalid inputs is finite, infeasible to try all of them

To maximize the chances of finding and fixing bugs, we need to choose a practically small number of inputs that, collectively,

- represent all possible inputs and
- are *most likely* to reveal bugs



Agenda

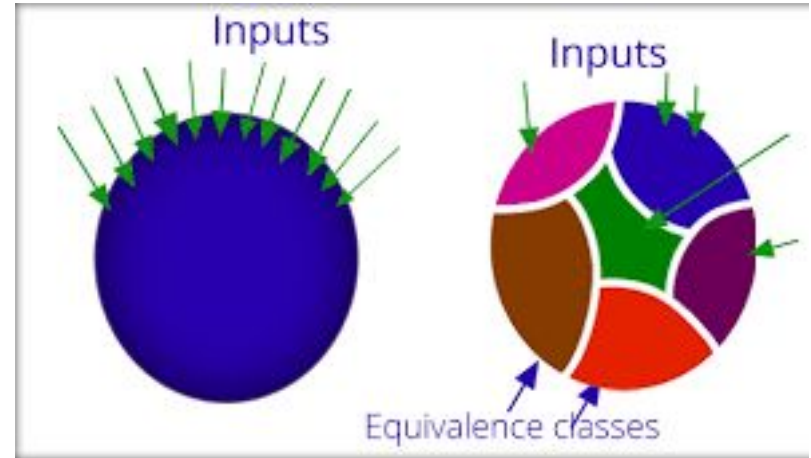
1. Choosing Test Inputs
2. Equivalence Partitions and Boundary Analysis
3. Reminders

<i>Testing</i>	
Advantages	Disadvantages
Provide a sense of complete testing and eradicates the need for exhaustive testing	Suitable only for range-wise and discrete values input data
Enables large domain of inputs or outputs coverage with a smaller subset selected from an equivalence class	Assumes that the data in the same equivalence class is processed in the same way by the system

Equivalence Partitions

The simplest approach for representing all possible inputs to a system with a practically small number of examples is to divide the input space into *equivalence partitions* (also known as equivalence classes) based on **both**

- domain knowledge
- software engineering knowledge (where do bugs like to hide?)



Equivalence Partitions

Equivalence partition = set of alternative input values where the software can reasonably be expected to behave equivalently (similarly)

This does **not** mean literally the same output for every input in the class!



Instead it means that if the software behaves correctly on one example input chosen from that equivalence class, it is reasonable to believe it will behave correctly on all the other inputs in the same class

Think about what kind of computation or processing is needed for different members of the class - if it's not very similar then it's not a true equivalence class and you need to rethink your partitioning

Simple Example

If we are testing a “box” intended to accept numbers from 1 to 10, then it might not be unreasonable to write ten test cases for all 10 valid input numbers

If we are testing a “box” intended to accept numbers from 1 to 1000000, then it *would* be unreasonable to write a million test cases for all 1000000 valid inputs!

1,000,000
1,000,000,000
1,000,000,000,000
1,000,000,000,000,000
1,000,000,000,000,000,000
1,000,000,000,000,000,000,000
1,000,000,000,000,000,000,000,000

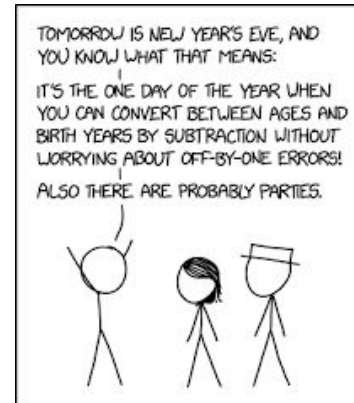
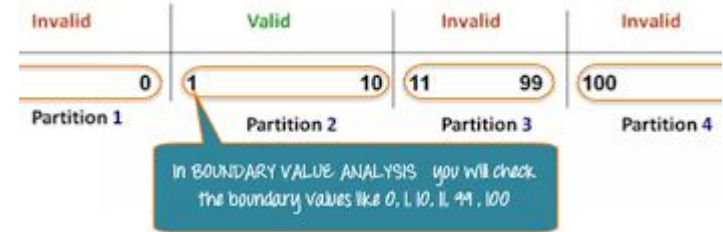
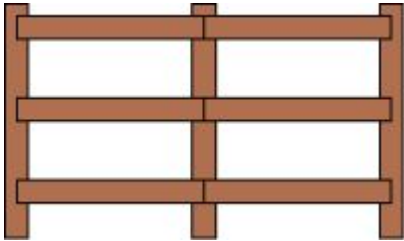
Instead, divide the test cases into three sets:

- 1) Equivalence partition with all valid inputs. Pick a single value from range 1 to 1000000 as a valid test case. If we select other values between 1 and 1000000 the result “should” also be correct (or also be incorrect)
- 2) Pick a single value for the partition with all values *below* the lower limit of the range, i.e., any value below 1
- 3) Pick a single value for the partition with all values *above* the upper limit, i.e., any value above 1000000

➤ Are we done yet?

Boundary Value Analysis (or just Boundary Analysis)

Equivalence partitions are typically extended with [boundary analysis](#) because input values at the extreme ends of the valid and invalid partitions are more likely to trigger bugs = “[off by one error](#)” or “[fencepost error](#)”

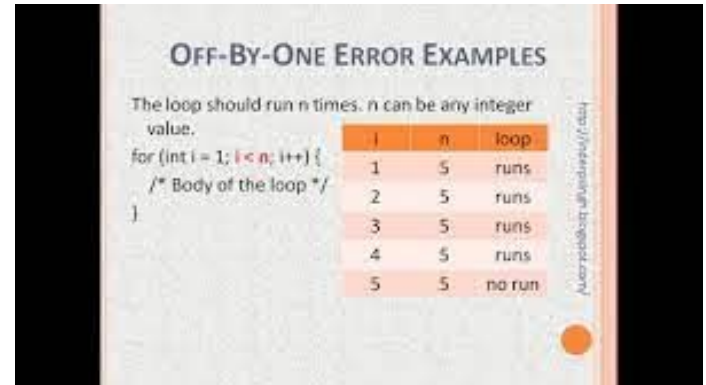


Boundary Value Analysis

- 1) Test cases with input exactly at the boundaries of the valid partition, i.e., values 1 and 1000000
- 2) Test data with values just below the extreme edges, i.e., values 0 and 999999
- 3) Test data with values just above the extreme edges, i.e., values 2 and 1000001

For integer domains, make sure to include zero, positive and negative examples, as well as the underlying primitive extremes

In this case, we already have 0 and some positive values, so we need to add some negative value, e.g., -1, plus MinInt and MaxInt



OFF-BY-ONE ERROR EXAMPLES

The loop should run n times. n can be any integer value.

```
for (int i = 1; i < n; i++) {  
    /* Body of the loop */  
}
```

i	n	loop
1	5	runs
2	5	runs
3	5	runs
4	5	runs
5	5	no run

<http://www.purpleboogym.com/>

Another Example

Consider a calculator app implemented in software, with a GUI as shown. The user clicks the buttons with a mouse, but cannot enter text into the display

In mathematics, the “+” operation takes two numeric operands

Say this software is tested by an external testing team separate from the developers, so they don't know how the app was implemented

Based on [domain knowledge](#), a tester might reasonably expect the “+” operation to behave similarly on all pairs of numbers < first operand, second operand >.

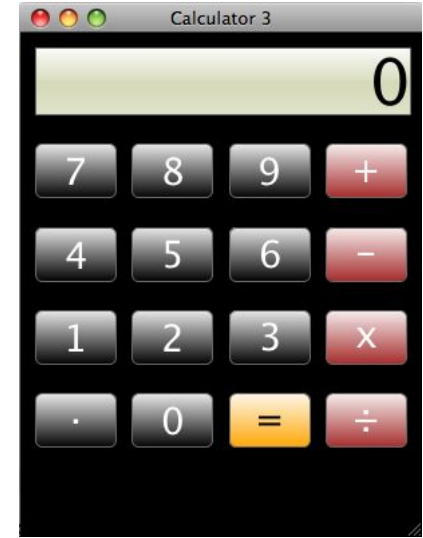


Continuing Example

So the tester tries an arbitrary pair of numbers:

< 3.141592653, 42 >

The calculator flashes 00000000. What might this mean?



Interpreting Test Outputs

Based on [software engineering knowledge](#), the tester might realize the flashing 00000000 could indicate:

- An error message because the software cannot handle numbers with more than N digits (e.g., 8)
- An error message because the software cannot handle mixed operands, i.e., one operand is floating point and the other is integer
- A bug
- Something else, e.g., the user has reached the limit of their “free trial” and now has to pay to continue using the calculator software

**WHEN ALL ELSE FAILS
READ THE INSTRUCTIONS**

Continuing Calculator Example

Let's ignore the “.” key (placeholder for version 2.0) and assume the “+” operator is currently intended only for integer operands

A tester with software engineering knowledge, not just domain knowledge, might realize that a calculator implementation (or any other software dealing with integers) could behave differently for positive integers, zero, and negative integers

Thus they test with five different equivalence classes:

- < pos int, pos int >
- < pos int, neg int >
- < pos int, 0 >
- < neg int, neg int >
- < 0, 0 >

Is this sufficient?



Software is not Mathematics

Although the “+” operator is mathematically commutative, that does not mean that the implementation (which could be buggy) is commutative

Or, more generally, we need to define equivalence partitions separately per parameter, and then extend to the cross-product

But notice the scaling problem with large numbers of parameters!

- < pos int, pos int >
- < pos int, neg int >
- < pos int, 0 >
- < neg int, pos int >
- < neg int, neg int >
- < neg int, 0 >
- < 0, pos int >
- < 0, neg int >
- < 0, 0 >



Extremes

Let's say the calculator is intended to handle operands only up to 8 digits (or to any other fixed number of digits)

This means there is a `MaxInt` and a `MinInt` (negative), probably specific to what the calculator GUI was designed to display rather than `MaxInt/MinInt` of the programming language, compiler, operating system, etc.

What additional tests are needed?

- `< xxx, MaxInt >`
- `< MaxInt, xxx >`
- `< xxx, MinInt >`
- `< MinInt, xxx >`
- `< MaxInt, MinInt >`
- `< MinInt, MaxInt >`



Invalid Inputs to Calculator Example



What if the user could enter text into the display window in addition to mousing the keys

How does that change the set of equivalence partitions that need to be tested?

Another Example

What are the
equivalence partitions?

How would we test this
program?

Design-your-burger

Please make your selection

Burger:

Cooked: ☐ rare ☐ medium ☐ well

Cheese: ☐

Lettuce: ☐

Tomato: ☐

Onion: ☐

Ketchup: ☐

Mustard: ☐

Mayo: ☐

Secret sauce: ☐



[Nutrition facts](#) [About the chef](#)

Required vs Optional Inputs

Do this ...

Title

Please select ... ▼

First name

Middle name(s) (optional)

Surname

... not this!

Title

Please select ... ▼

First name

* Middle name(s)

Surname

For some applications, equivalence classes need to address optional as well as required inputs, with or without default values

What are the equivalence partitions for this example?

How would we test this program?

Dependencies between Inputs

Distinct parameters may be related to each other in forming equivalence classes

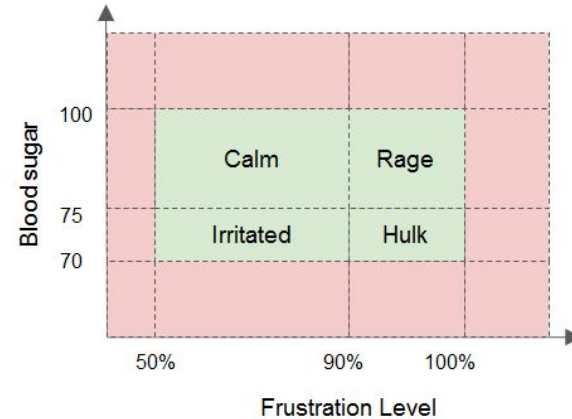
Let's say we had a function that took two variables, blood sugar level and level of frustration, to calculate "mood"



Example

Assume valid inputs are
Blood sugar 70 - 100
Frustration is 50% - 100%

For $50 \leq F \leq 90$
 $75 \leq BS \leq 100$: Calm
 $BS < 75$: Irritated
For $F > 90$
 $75 \leq BS \leq 100$: Rage
 $BS < 75$: [Hulk Smash](#)

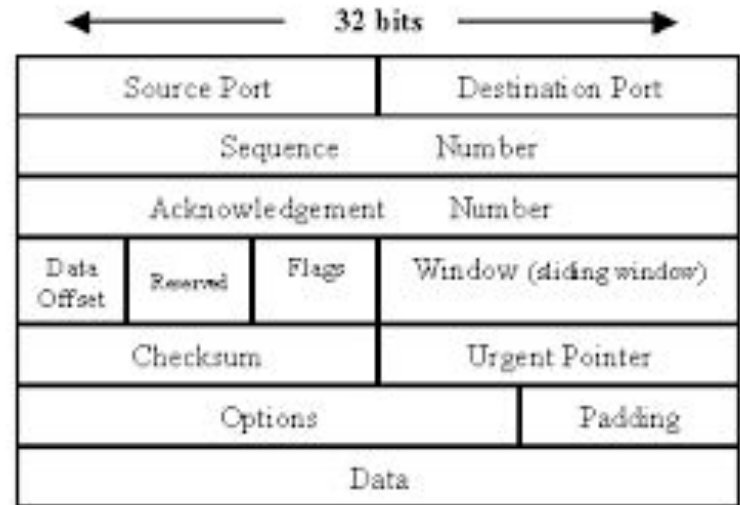


More Dependencies between Inputs

Equivalence classes may cut across multiple inputs:

Sometimes the value of one input restricts valid values for other inputs. For example, date of birth and age should be consistent (valid) or not (invalid)

Sometimes the value of one (small) input is intended as a sanity check on another (larger, more complicated) input, such as the [checksum in the network packet](#)

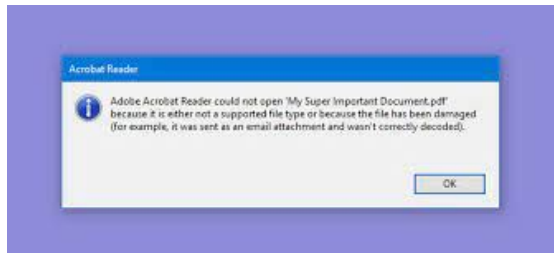


System to System Inputs

At the full system and service levels, inputs are typically going to be numbers, strings (text), tuples (packets, buffers, json, xml), and files

File inputs can:

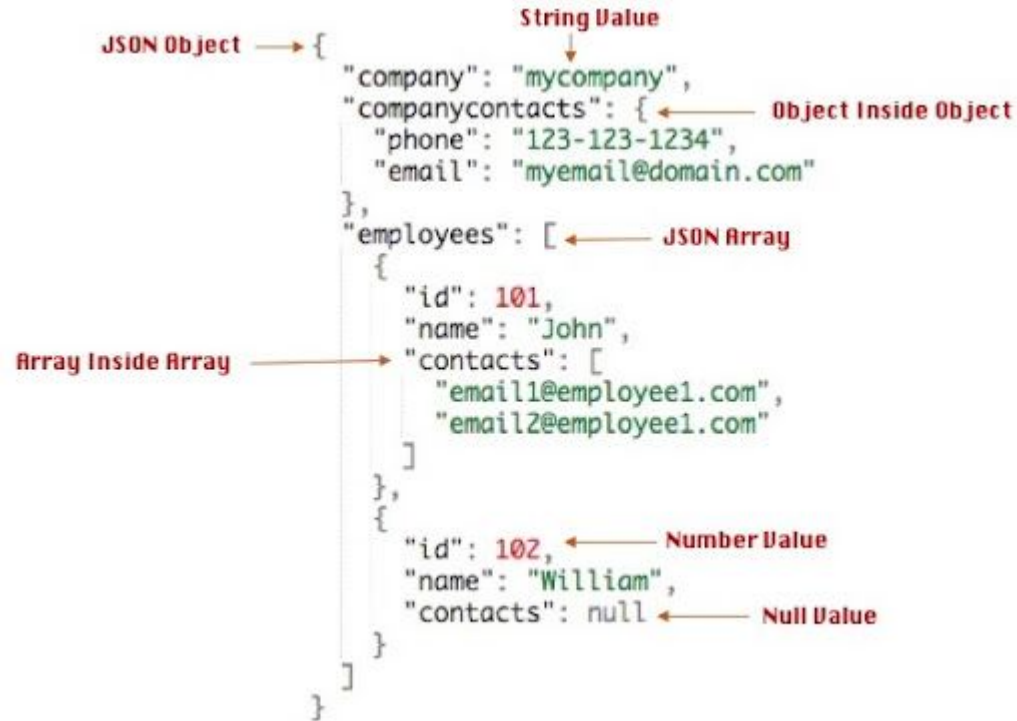
- Exist or not
- Be below min size or above max size
- Be readable, writeable, executable by current user (permissions)
- Correct, corrupted, garbled, etc. data and metadata formats - images, audio, video, ...



'Systems' here includes Services with APIs

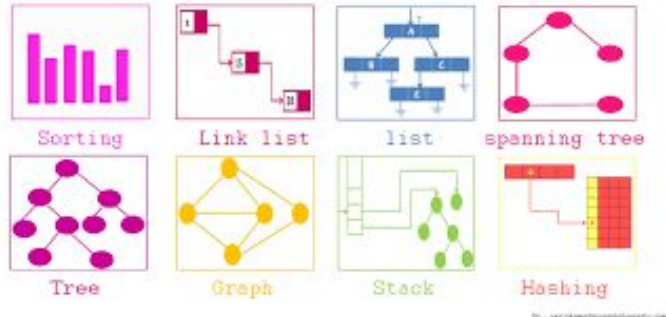
What might be the equivalence classes for inputs with this format?

What are some plausible invalid formats?



Internal Inputs

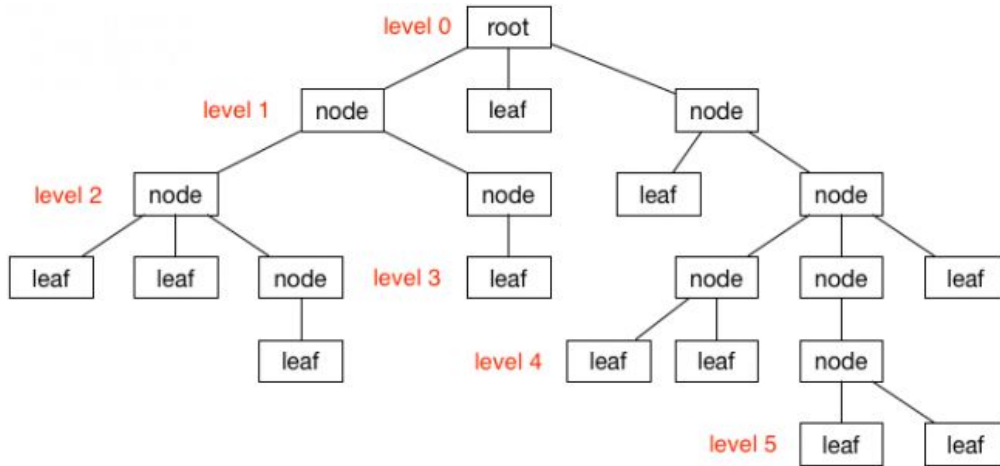
Inputs/outputs for unit tests can be arbitrary in-memory data structures, so equivalence classes are more complex - and inputs and expected outputs (for assertion comparison) are more complicated to construct



For example, container data structures with content elements:

- In or not in container
- Empty or full container (or min/max number of elements)
- Duplicate elements may or may not be allowed
- Ordered or organized according to some metric, or not

More Internal Inputs



Specific kinds of containers, e.g., tree
- root node, interior node, leaf node,
null tree, tree with exactly one node,
“full” tree, balanced vs. unbalanced,
various specialty trees and graphs

Test fixtures need to construct tree
nodes, cyclic and acyclic graphs, etc.
to use as test inputs

Later in semester we will discuss other approaches to test input generation,
such as [fuzzing](#)

Agenda

1. Choosing Test Inputs
2. Equivalence Partitions and Boundary Analysis
3. Reminders



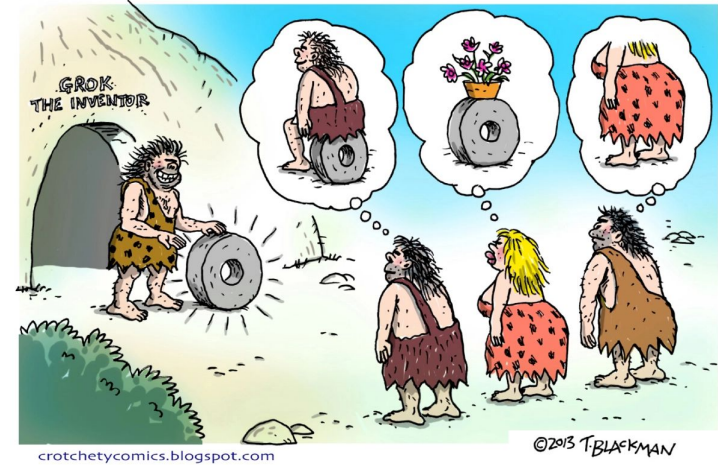
Deadlines

[Team First Iteration](#) due Friday October 18

[Team First Iteration Demo](#) due Friday October 25

First Assessment Friday October 25th 12:01am
through Monday October 28th 11:59pm -

We will discuss what you should expect when closer,
just one hint now: don't neglect the Reading assignments!



Thursday

finish anything we did not finish today

Unit Testing and Mocking, with demo



Ask Me Anything

